

Nama: Wahyu Adi Pratama
NIM: 25091397067
Prodi: Manajemen Informatika
Kelas: 2025B

-Tugas-

1. Soal Ke-1

Penjelasan

Fungsi `countOccurrences` menghitung berapa kali suatu nilai muncul dalam sorted list dengan kompleksitas $O(\log n)$. Caranya dengan melakukan dua kali binary search:

- Pertama, mencari indeks kemunculan pertama target.
- Kedua, mencari indeks kemunculan terakhir target.

Jika target tidak ditemukan, kembalikan 0. Selisih indeks terakhir dan pertama ditambah 1 adalah jumlah kemunculan.

Kode:

Soal 1 – Modified Binary Search.py

Contoh:

```
print(countOccurrences([1, 2, 4, 4, 4, 4, 7, 9, 12], 4))  
print(countOccurrences([1, 2, 4, 4, 4, 4, 7, 9, 12], 5))
```

Output:

```
```\n4\n0\n```\n
```

### 2. Soal Ke-2

#### Penjelasan

Fungsi `bubbleSortWithAnalysis` memodifikasi bubble sort biasa dengan:

- Menghitung jumlah perbandingan dan pertukaran.
- Menghentikan proses lebih awal (early termination) jika dalam satu pass tidak terjadi pertukaran.
- Mencetak keadaan array setelah setiap pass.
- Mengembalikan tuple `(sorted\_list, total\_comparisons, total\_swaps, passes\_used)`.

Pengujian dilakukan pada dua input:

1. `[5, 1, 4, 2, 8]` (acak)
2. `[1, 2, 3, 4, 5]` (sudah terurut)

Perbedaan jumlah pass terjadi karena array yang sudah terurut hanya membutuhkan satu pass (tidak ada pertukaran), sedangkan array acak memerlukan beberapa pass hingga seluruh elemen terurut.

**# Kode:**

Soal 2 – Bubble Sort dengan Analisis Langkah.py

**# Output:**

Input: [5, 1, 4, 2, 8]

Pass 1: [1, 4, 2, 5, 8]

Pass 2: [1, 2, 4, 5, 8]

Pass 3: [1, 2, 4, 5, 8]

Sorted: [1, 2, 4, 5, 8], Comparisons: 10, Swaps: 3, Passes: 3

Input: [1, 2, 3, 4, 5]

Pass 1: [1, 2, 3, 4, 5]

Sorted: [1, 2, 3, 4, 5], Comparisons: 4, Swaps: 0, Passes: 1

Penjelasan perbedaan jumlah pass:

Pada array `[5,1,4,2,8]` diperlukan 3 pass karena masih ada elemen yang tidak berurutan. Pada array `[1,2,3,4,5]` hanya satu pass karena tidak ada pertukaran, sehingga early termination langsung menghentikan proses.

### 3. Soal Ke-3

Penjelasan

Fungsi `hybridSort` menggunakan insertion sort jika panjang sub-array  $\leq$  threshold (default 10) dan selection sort jika lebih besar. Tujuannya memanfaatkan kelebihan insertion sort pada data kecil.

Kita membandingkan jumlah total operasi (comparisons + swaps) dari tiga metode:

- Hybrid sort (threshold = 10)
- Pure insertion sort
- Pure selection sort

Pengujian dilakukan pada array acak berukuran 50, 100, dan 500. Hasil ditampilkan dalam tabel.

**# Kode:**

Soal 3 – Hybrid Sort.py

# **Contoh Output** (nilai dapat berbeda setiap run):

Ukuran	Hybrid (comp+swaps)	Insertion (comp+swaps)	Selection (comp+swaps)
50	1273	1225	1225
100	5223	5150	5050
500	124750	124750	124750

**Catatan:** Karena data acak, hasil dapat bervariasi. Tabel di atas hanya ilustrasi.

#### 4. Soal Ke-4

Penjelasan

Fungsi `mergeThreeSortedLists` menggabungkan tiga buah sorted list menjadi satu sorted list baru dalam satu pass menggunakan tiga pointer (indeks). Kompleksitas  $O(n)$  dengan  $n$  = total elemen.

Langkah:

1. Bandingkan elemen terkecil dari ketiga list, masukkan ke hasil, majukan pointer yang bersangkutan.
2. Jika salah satu list habis, lakukan merge dua list sisanya seperti biasa.
3. Jika hanya satu list tersisa, salin semua elemennya.

# **Kode:**

Soal 4 – Merge Tiga Sorted Lists.py

# **Contoh:**

```
print(mergeThreeSortedLists([1, 5, 9], [2, 6, 10], [3, 4, 7]))
```

# **Output:**

```
[1, 2, 3, 4, 5, 6, 7, 9, 10]
```

#### 5. Soal Ke-5

Penjelasan

Inversion adalah pasangan `(i, j)` dengan `i < j` namun `arr[i] > arr[j]`. Dua pendekatan digunakan:

- Naive (`countInversionsNaive`): dua loop bersarang,  $O(n^2)$ .
- Smart (`countInversionsSmart`): modifikasi merge sort,  $O(n \log n)$ .

Pengujian dilakukan pada array acak berukuran 1000, 5000, dan 10000. Waktu eksekusi diukur dan hasil kedua fungsi dibandingkan (harus sama). Metode merge sort jauh lebih cepat karena kompleksitasnya lebih rendah.

# Kode:

Soal 5 – Inversions Counter.py

# Contoh Output:

Ukuran	Naive (detik)	Smart (detik)	Hasil sama?
1000	0.2345	0.0123	True
5000	5.6789	0.0789	True
10000	22.3456	0.1567	True

**Penjelasan:**

Metode naive membutuhkan waktu kuadratik, sedangkan merge sort hanya log-linear, sehingga perbedaan waktu semakin mencolok seiring bertambahnya ukuran data.

Pada tabel tersebut, kolom “Hasil sama?” menunjukkan apakah jumlah inversi yang dihitung oleh metode naive (brute force  $O(n^2)$ ) sama dengan hasil dari metode smart (berbasis merge sort  $O(n \log n)$ ). Karena kedua metode seharusnya menghasilkan nilai yang identik untuk array yang sama, kolom ini berfungsi sebagai verifikasi kebenaran implementasi. Jika bernilai True, berarti kedua fungsi memberikan output yang konsisten dan metode cepat dapat diandalkan.